

CIS 25
Final Presentation
Professor Lakisha
Brooks

Personal Finance **Budget Tracker**

01
**Project
overview**

02
**Data Structures
& OOP**

03
**Data
Management**

04
**Code
Snippets**

05
Menu Code

06
Demonstration

07
**Possible
Enhancements**

08
Conclusion

Table of contents

01

Project

overview

Personal Finance/Budget Tracker

- Allows user to record, categorize, and manage their income and expenses
 - Stores and analyzes transaction data
- Creates summaries on user spending history
- Implements data management and ensures data is stored between sessions
 - Sorts transactions based on posting date
 - Generates reports using algorithm

Personal Finance/ Budget Tracker

02

Data Structures and OOP

Transaction Class

```
class Transaction {
public:
    std::string date;
    std::string type; //either income or expense type
    std::string category;
    std::string description;
    double amount;

    //constructor for class
    Transaction(std::string d, std::string t, std::string cat, std::string desc, double amt)
        : date(d), type(t), category(cat), description(desc), amount(amt) {}

    //method to convert transaction into string for file storage in csv
    std::string to_csv() const {
        return date + "," + type + "," + category + "," + description + "," + std::to_string(amount);
    }
};
```

Account Class

```
class Account {
public:
    std::string name;
    double balance;
    std::vector<Transaction> transactions;

    //constructor
    Account(std::string n, double bal = 0.0) : name(n), balance(bal) {}

    //method to add transaction and update balance
    void add_transaction(const Transaction& t) {
        transactions.push_back(t);
        if (t.type == "Income") {
            balance += t.amount;
        } else {
            balance -= t.amount;
        }
    }

    //method to convert account to string for file storage
    std::string to_string() const {
        return name + "," + std::to_string(balance);
    }
};
```

FinanceTracker Class

```
class FinanceTracker {
private:
    std::vector<Account> accounts;
    const std::string DATA_FILE = "finance_data.txt";
    //helper funcs
    void save_data();
    void load_data();

public:
    FinanceTracker();
    ~FinanceTracker(); //destructor to make sure data is saved when exited

    void add_account(const std::string& name, double initial_balance = 0.0);
    Account* get_account(const std::string& name);

    void record_transaction(const std::string& account_name, const std::string& type,
                           const std::string& category, const std::string& description,
                           double amount);

    void display_summary() const;
    void generate_report(const std::string& account_name) const;

    //funct to get date in YYYY-MM-DD
    std::string get_current_date();
};
```


FinanceTracker Class

```
class FinanceTracker {
private:
    std::vector<Account> accounts;
    const std::string DATA_FILE = "finance_data.txt";
    //helper funcs
    void save_data();
    void load_data();

public:
    FinanceTracker();
    ~FinanceTracker(); //destructor to make sure data is saved when exited

    void add_account(const std::string& name, double initial_balance = 0.0);
    Account* get_account(const std::string& name);

    void record_transaction(const std::string& account_name, const std::string& type,
                           const std::string& category, const std::string& description,
                           double amount);

    void display_summary() const;
    void generate_report(const std::string& account_name) const;

    //funct to get date in YYYY-MM-DD
    std::string get_current_date();
};
```

03

Data Management

Input and Output Files

```
const std::string DATA_FILE = "finance_data.txt";
```

```
std::ifstream infile(DATA_FILE);
```

```
if (!infile.is_open()) { //file does not exist yet so we start with empty file
```

```
std::ofstream outfile(DATA_FILE);
```

```
if (!outfile.is_open()) {
```

```
    std::cerr << "error could not open file." << std::endl;
```

Helper Functions - save data

```
void FinanceTracker::save_data() { //saves all account and transaction data to file
    std::ofstream outfile(DATA_FILE);
    if (!outfile.is_open()) {
        std::cerr << "error could not open file." << std::endl;
        return;
    }

    outfile << accounts.size() << std::endl; //writes number of accs

    for (const auto& acc : accounts) { //writes each account details and transactions such as name, balance, transaction count
        outfile << acc.name << "," << acc.balance << "," << acc.transactions.size() << "\n";
        for (const auto& t : acc.transactions) {
            outfile << t.to_csv() << std::endl;
        }
    }
    outfile.close();
    std::cout << "data saved successfully.\n";
}
```

Helper Functions - load data 1

```
void FinanceTracker::load_data() { //loads all account and transaction data from file
    std::ifstream infile(DATA_FILE);
    if (!infile.is_open()) { //file does not exist yet so we start with empty file
        return;
    }
    std::string line;
    size_t num_accounts = 0;
    if (std::getline(infile, line)) { //reads number of accounts
        try { num_accounts = std::stoul(line);
        } catch (...) {
            std::cerr << "warning: data file error or empty.\n"; //incase of malformed files
            infile.close();
            return;
        }
    }
}
```

Helper Functions - load data 2

```
for (size_t i = 0; i < num_accounts; ++i) { //read each account and transactions
    if (std::getline(infile, line)) { //reads header
        std::stringstream ss(line);
        std::string name_str, balance_str, num_transactions_str;
        std::getline(ss, name_str, ',');
        std::getline(ss, balance_str, ',');
        std::getline(ss, num_transactions_str, ',');
        double balance = std::stod(balance_str);
        size_t num_transactions = std::stoul(num_transactions_str);
        Account acc(name_str, balance);

        for (size_t j = 0; j < num_transactions; ++j) { //reads each transaction
            if (std::getline(infile, line)) {
                std::stringstream ts(line);
                std::string date, type, category, description, amount_str;
                std::getline(ts, date, ',');
                std::getline(ts, type, ',');
                std::getline(ts, category, ',');
                std::getline(ts, description, ',');
                std::getline(ts, amount_str, ',');
                double amount = std::stod(amount_str);
                Transaction t(date, type, category, description, amount);
                //add transaction w out updating balance
                acc.transactions.push_back(t);
            }
        }
        accounts.push_back(acc);
    }
}

infile.close();
}
```

Constructors

```
FinanceTracker::FinanceTracker() { //constructor loads existing data when created  
|   load_data();  
}
```

```
FinanceTracker::~~FinanceTracker() { //deconstructor saves data when program exit  
|   save_data();  
}
```

04

Code Snippets

Add_Account & Get_Account

```
void FinanceTracker::add_account(const std::string& name, double initial_balance) { //adds new acc to tracker
    if (get_account(name) != nullptr) {
        std::cout << "account '" << name << "' already exists.\n";
        return;
    }
    accounts.emplace_back(name, initial_balance);
    std::cout << "account '" << name << "' created with initial balance $" << initial_balance << ".\n";
}
```

```
Account* FinanceTracker::get_account(const std::string& name)
{ //gets pointer to an account by name. only for internal use
    for (auto& acc : accounts) {
        if (acc.name == name) {
            return &acc;
        }
    }
    return nullptr;
}
```

Record_Transaction & Display_Summary

```
//records new transaction into specific account.
void FinanceTracker::record_transaction(const std::string& account_name, const std::string& type, const std::string& category, const std::string& description, double amt) {
    Account* acc = get_account(account_name);
    if (!acc) {
        std::cout << "error: account '" << account_name << "' not found.\n";
        return;
    }
    std::string date = get_current_date();
    Transaction t(date, type, category, description, amt);
    acc->add_transaction(t);
    std::cout << type << " of $" << amt << " recorded in " << account_name << ".\n";
}

void FinanceTracker::display_summary() const { //displays current balance for all accounts

    double total_net_worth = 0.0;
    std::cout << "\nfinancial summary:\n";
    for (const auto& acc : accounts) {
        std::cout << "account: " << acc.name << " --- balance: $" << acc.balance << "\n";
        total_net_worth += acc.balance;
    }
    if (total_net_worth == 0.0 && accounts.empty()) {
        std::cout << "no accounts found.\n";
    }
    std::cout << "total net worth: $" << total_net_worth << std::endl;
}
```

Generate_Report & Sort

```
void FinanceTracker::generate_report(const std::string& account_name) const { //creates report with sorted transactions and category totals for specific account
    const Account* acc = nullptr;
    for (const auto& a : accounts) {
        if (a.name == account_name) {
            acc = &a;
            break;
        }
    }
    if (!acc)
    {
        std::cout << "error: account '" << account_name << "' not found.\n";
        return;
    }

    std::vector<Transaction> sorted_transactions = acc->transactions;
    std::sort(
        sorted_transactions.begin(),
        sorted_transactions.end(), //sorts transactions by date (uses string comparison)
        [](const Transaction& a, const Transaction& b) {
            return a.date < b.date;
        }
    );
};
```

Report

```
std::cout << "\ntransaction report for " << acc->name << std::endl;
std::cout << "current balance: $" << acc->balance << std::endl;
std::cout << "category summary\n";
std::map<std::string, double> cat_totals;
double total_income = 0.0;
double total_expense = 0.0;

for (const auto& t : sorted_transactions)
{
    if (t.type == "income")
    {
        cat_totals["income: " + t.category] += t.amount;
        total_income += t.amount;
    } else {
        cat_totals["expense: " + t.category] += t.amount;
        total_expense += t.amount;
    }
}

for (const auto& pair : cat_totals) {
    std::cout << pair.first << ": $" << pair.second << std::endl;
}

std::cout << "total income: $" << total_income << std::endl;
std::cout << "total expense: $" << total_expense << std::endl;
std::cout << "net flow: $" << (total_income - total_expense) << "\n";

std::cout << "detailed transactuions by date\n";
for (const auto& t : sorted_transactions) {
    std::cout << "[" << t.date << "]" | "
    |
    |
    |
    << (t.type == "income" ? "INCOME" : "EXPENSE") << " | "
    << "$" << t.amount << " | " << t.category << " | "
    << t.description << "\n";
}
}
```

get_current_date

```
std::string FinanceTracker::get_current_date() { //gets current date as string in YYYYMMDD format
    std::time_t now = std::time(nullptr);
    std::tm* ltm = std::localtime(&now);
    std::stringstream ss;
    ss << 1900 + ltm->tm_year << "-"
        << std::setfill('0') << std::setw(2) << 1 + ltm->tm_mon << "-"
        << std::setfill('0') << std::setw(2) << ltm->tm_mday;
    return ss.str();
}
```

05

Menu Code

```

int main() {
    FinanceTracker tracker; //initializes and then loads data
    tracker.display_summary(); //checks if loaded successfully

    int choice;
    do {
        display_menu();
        std::cout << "enter choice: ";
        if (!(std::cin >> choice)) {
            std::cout << "invalid input.\n";
            std::cin.clear(); //clears error flags and discards bad input
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            continue;
        }
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

        //menu
        switch (choice) {
            case 1:
                handle_add_account(tracker);
                break;
            case 2:
                handle_record_transaction(tracker);
                break;
            case 3:
                tracker.display_summary();
                break;
            case 4:
                handle_generate_report(tracker);
                break;
            case 5:
                std::cout << "exiting application. data will be saved.\n"; //deconstruction happens automatically when program exited
                break;
            default:
                std::cout << "invalid choice. please try again.\n";
        }
    } while (choice != 5);
    return 0;
}

```

```

//displays menu for gui
void display_menu() {
    std::cout << "Bella's Personal Finance Tracker\n";
    std::cout << "1. Add New Account\n";
    std::cout << "2. Record Transaction (Income/Expense)\n";
    std::cout << "3. View Account Summary\n";
    std::cout << "4. Generate Report\n";
    std::cout << "5. Exit\n";
}

//handles user input for adding a new account.
void handle_add_account(FinanceTracker& tracker) {
    std::string name;
    double balance = 0.0;
    std::cout << "\nTO ADD NEW ACCOUNT:\n";
    std::cout << "Enter account name (Checking, Savings): ";
    std::getline(std::cin, name);

    std::cout << "Enter initial balance: $";
    if (!(std::cin >> balance)) {
        std::cout << "invalid balance. cannot create account\n";
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        return;
    }
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    tracker.add_account(name, balance);
}

```



```

//handles user input for transaction
void handle_record_transaction(FinanceTracker& tracker) {
    std::string account_name, type, category, description;
    double amount;
    int type_choice;
    std::cout << "\nTO RECORD TRANSACTION\n";
    std::cout << "Enter account name for transaction: ";
    std::getline(std::cin, account_name);

    if (!tracker.get_account(account_name)) { //check account exists first
        std::cout << "Account not found. Please create account first.\n";
        return;
    }
    std::cout << "Transaction type (1 - Income, 2 - Expense): ";

    if (!(std::cin >> type_choice) || (type_choice != 1 && type_choice != 2)) {
        std::cout << "Invalid choice. Transaction failed.\n";
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        return;
    }
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

    type = (type_choice == 1) ? "Income" : "Expense";

    std::cout << "Enter category (ex: Salary, Rent, Groceries): ";
    std::getline(std::cin, category);
    std::cout << "Enter description: ";
    std::getline(std::cin, description);
    std::cout << "Enter amount: $";

    if (!(std::cin >> amount) || amount <= 0) {
        std::cout << "Invalid amount. Transaction failed.\n";
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        return;
    }
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    tracker.record_transaction(account_name, type, category, description, amount);
}

```

```
//handles input 4 generating a specific account report.  
void handle_generate_report(FinanceTracker& tracker) {  
    std::string account_name;  
    std::cout << "\nGENERATE REPORT\n";  
    std::cout << "Enter account name for report: ";  
    std::getline(std::cin, account_name);  
    tracker.generate_report(account_name);  
}
```

06

Demonstration

```
isabellalynnabetong@Isabellas-MacBook-Air final project % ./tracker
```

```
financial summary:
```

```
no accounts found.
```

```
total net worth: $0
```

```
Bella's Personal Finance Tracker
```

1. Add New Account
2. Record Transaction (Income/Expense)
3. View Account Summary
4. Generate Report
5. Exit

```
enter choice: 1
```

TO ADD NEW ACCOUNT:

Enter account name (Checking, Savings): Checkings

Enter initial balance: \$5

account 'Checkings' created with initial balance \$5.

Bella's Personal Finance Tracker

1. Add New Account
2. Record Transaction (Income/Expense)
3. View Account Summary
4. Generate Report
5. Exit

enter choice: 2

TO ADD NEW ACCOUNT:

Enter account name (Checking, Savings): Checkings

Enter initial balance: \$5

account 'Checkings' created with initial balance \$5.

Bella's Personal Finance Tracker

1. Add New Account
2. Record Transaction (Income/Expense)
3. View Account Summary
4. Generate Report
5. Exit

enter choice: 2

TO RECORD TRANSACTION

Enter account name for transaction: Checkings

Transaction type (1 – Income, 2 – Expense): 1

Enter category (ex: Salary, Rent, Groceries): Job Income

Enter description: Gold Bean

Enter amount: \$1000

Income of \$1000 recorded in Checkings.

Bella's Personal Finance Tracker

1. Add New Account
2. Record Transaction (Income/Expense)
3. View Account Summary
4. Generate Report
5. Exit

enter choice: 3

financial summary:

account: Checkings --- balance: \$1005

total net worth: \$1005

Bella's Personal Finance Tracker

1. Add New Account
2. Record Transaction (Income/Expense)
3. View Account Summary
4. Generate Report
5. Exit

enter choice: 4

GENERATE REPORT

Enter account name for report: Checkings

transaction report for Checkings

current balance: \$1005

category summary

expense: Job Income: \$1000

total income: \$0

total expense: \$1000

net flow: \$-1000

detailed transactuions by date

[2025-12-10]		EXPENSE		\$1000		Job Income		Gold Bean
--------------	--	---------	--	--------	--	------------	--	-----------

Bella's Personal Finance Tracker

1. Add New Account
2. Record Transaction (Income/Expense)
3. View Account Summary
4. Generate Report
5. Exit

enter choice: 5

exiting application. data will be saved.
data saved successfully.

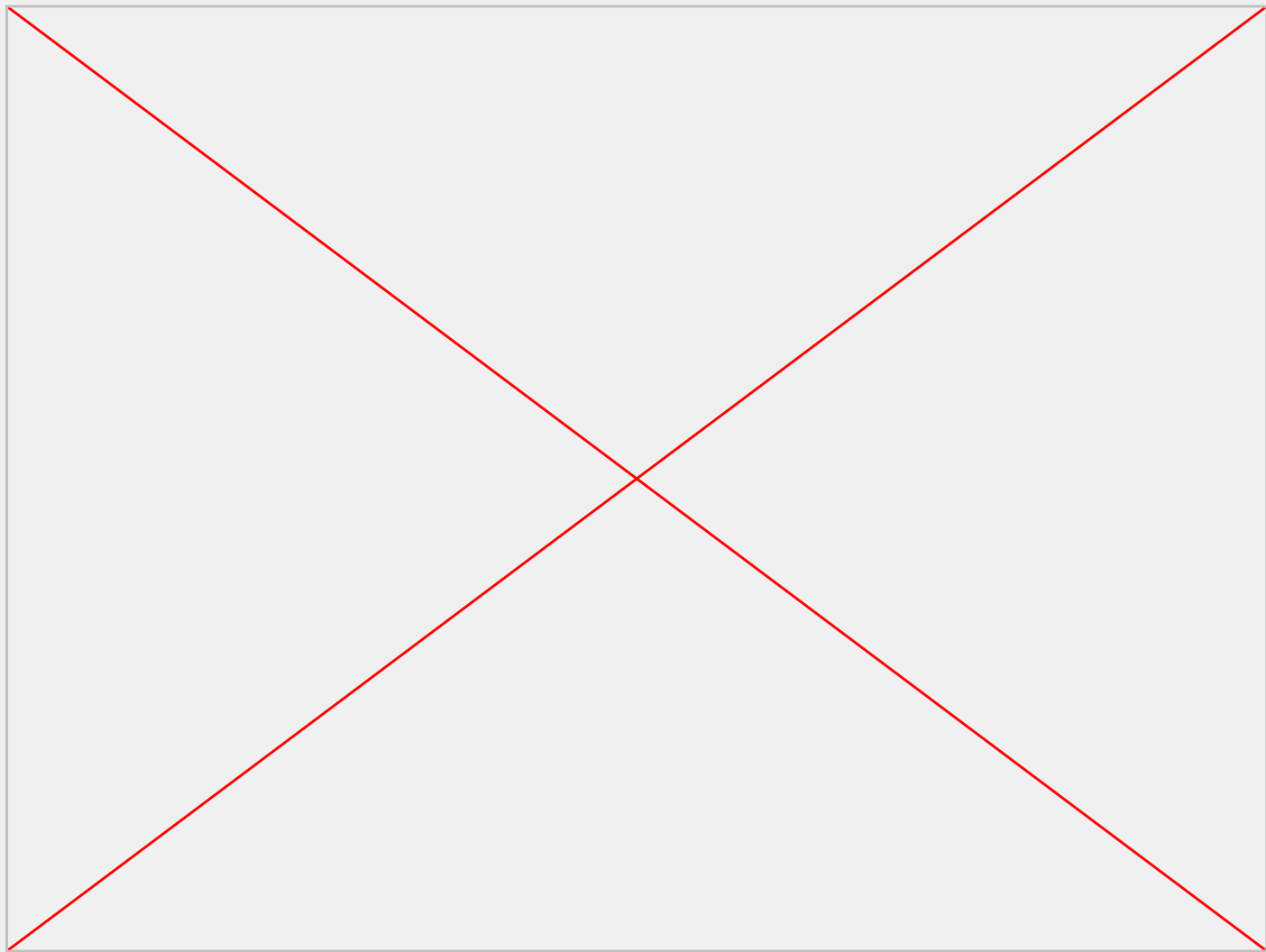
inal project > ≡ finance_data.txt

1 1

2 Checkings,1005,1

3 2025-12-10,Income,Job Income,Gold Bean,1000.000000

4



06

Enhancements

01

GUI vs CLI

Currently runs on Command Line Input. Could be improved by implementing backend/frontend GUI with buttons for better viewing
Cons: Difficult to implement with C++ libraries

02

Database -SQLite

Replace File I/O with SQLite for more robust data management

03

Advanced Features

Could Implement monthly/quartile comparison analysis

Possible Future Enhancements

06

Conclusion

Do you have any questions?

Github link:

https://github.com/isabella921/CIS25_Fall2025/

Youtube link:

<https://youtu.be/tnOfI2AQpN8>

CREDITS: This presentation template was created by [Slidesgo](#), and includes icons, infographics & images by [Freepik](#)

Please keep this slide for attribution

Thank
you